

Hospital Patient Care Analytics Pipeline

An end-to-end data engineering solution for multi-specialty hospital operations & patient-care analytics

STUDENT NAME	STUDENT ID
Chetan Kalagarla	2300033450

INGESTION

QUALITY CHECKS

TRANSFORMATION

SQLITE STORAGE

RISK ANALYTICS

Data note — this case study uses fully synthetic records to demonstrate the engineering workflow. The demonstration risk score is not a clinical model and must not be used for medical decision-making.

01 • PROJECT OVERVIEW

Purpose & Implementation Scope

This project designs an end-to-end data engineering solution for a multi-specialty hospital, built to collect, process, store and analyze data arriving from multiple independent healthcare systems. The pipeline supports two goals at once: sharpening day-to-day operational efficiency (such as patient waiting times) and enabling structured, repeatable patient-care analytics.

End-to-End Implementation Scope



Core Objectives

- Create a unified, patient-level analytical dataset from five source systems.
- Measure and analyze waiting-time performance across departments.
- Transform raw measurements into useful, analysis-ready features.
- Generate simple, rule-based risk flags to support further clinical review.
- Maintain basic pipeline-quality and monitoring metrics throughout.

Scope note. The architecture follows a conventional healthcare data-engineering pattern: source systems feed ingestion, validation and transformation stages, followed by analytical storage and downstream reporting.

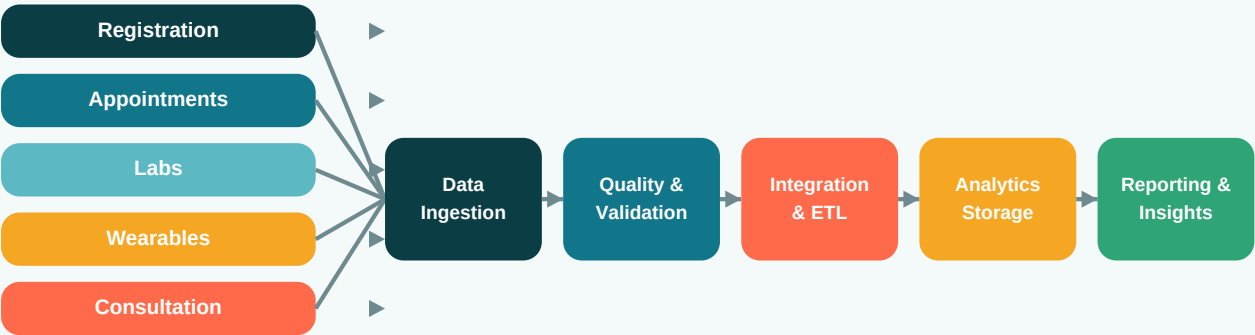
02 · PROBLEM ANALYSIS

Data Sources & Integration Key

The hospital receives information from five independent systems. The first engineering task is to identify the source data, understand its operational role, and define a common key — **patient_id** — that lets every table be joined into one analytical view.

Source System	Representative Fields	Engineering Purpose
Patient Registration	patient_id, age, gender, department	Patient master information
Appointment Scheduling	appointment_date, waiting_minutes, status	Patient-flow & waiting-time analysis
Laboratory Reports	hemoglobin, glucose, abnormal flag	Laboratory indicators
Wearable Devices	heart_rate, oxygen_sat, temperature	Vital-sign indicators
Doctor Consultation	symptom_score, consultation_minutes, follow_up	Consultation workflow indicators

End-to-End Data Engineering Structure



Five source systems converge through ingestion, validation and integration into analytics-ready storage and reporting.

03 · DATA PREPARATION

Synthetic Data, Quality Checks & Integration

The notebook creates small synthetic datasets to represent each source system — this keeps the demonstration fully reproducible while avoiding the use of any real patient information. Pandas, NumPy, Matplotlib and SQLite form the core toolset.

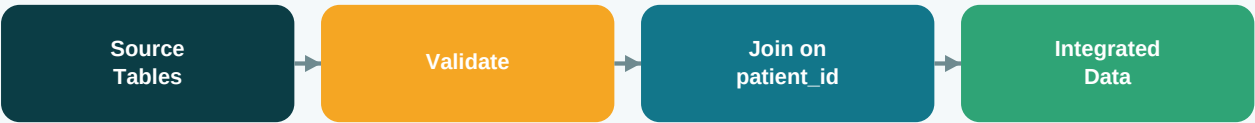
Notebook Implementation Sequence

- 1 Import Pandas, NumPy, Matplotlib and SQLite
- 2 Create patient registration records
- 3 Create appointment records with waiting times
- 4 Create laboratory measurements and abnormal-result indicators
- 5 Create wearable vital-sign measurements
- 6 Create doctor consultation indicators
- 7 Validate the source data before integration

Data-Quality Checks

Validation Check	Purpose
Row count	Confirms that expected records exist in each source.
Duplicate count	Identifies repeated records that may distort analytics.
Missing-value count	Highlights incomplete data before transformation.
Patient ID consistency	Ensures that records can be connected across systems.

Integration Strategy



This stage is the foundation of the pipeline: reliable analytics depend on consistent identifiers, basic validation and controlled integration before any downstream analysis.

04 · TRANSFORMATION

Feature Engineering & Storage Layer

After integration, the notebook derives additional features from the raw fields. These engineered features make the dataset more useful for operational analysis and for the demonstration risk analytics described later in this report.

Feature	Logic / Interpretation
age_group	Classifies patients into Young, Middle or Senior groups.
long_wait_flag	Flags waiting time of 45 minutes or more.
low_oxygen_flag	Flags oxygen saturation below 94%.
high_heart_rate_flag	Flags heart rate above 100.
high_glucose_flag	Flags glucose above 140.
high_symptom_flag	Flags symptom score of 7 or higher.
risk_score	Adds selected demonstration risk indicators.
risk_level	Groups the demonstration score into Low, Medium or High.

Storage Layer

The processed analytics table is written to a local SQLite database named **hospital_analytics.db**. The notebook then reads the stored table back using SQL, demonstrating a complete write-and-read storage cycle. The final analytics-ready data is also exported as **hospital_patient_analytics.csv**.



Risk-scoring limitation. The risk score is a simple rule-based demonstration created to show feature engineering and analytical flag generation. It has not been clinically validated and must not be interpreted as a medical diagnosis or treatment recommendation.

05 · ANALYTICS & MONITORING

Operational Analytics, Risk Analytics & Monitoring

The integrated dataset powers both operational and risk-focused analytical outputs. The notebook also produces monitoring metrics so that basic pipeline quality can be reviewed at a glance.

Operational Analytics

- Calculate the overall average patient waiting time.
- Calculate average waiting time by department.
- Visualize department-level waiting performance using a bar chart.

Patient-Risk Analytics

- Count patients by Low, Medium and High demonstration risk level.
- Create a high-risk review table with selected analytical indicators.
- Use the output as an analytics flag for further review — not a clinical decision.

Pipeline Monitoring Metrics

Metric	What It Monitors
Source patient rows	Volume of the registration source.
Integrated rows	Number of records after source integration.
Null values	Incomplete values in the integrated dataset.
Duplicate rows	Repeated integrated records.
High-risk records	Number of records receiving the demonstration high-risk flag.

Analytics Flow



06 • SUMMARY

Complete Implementation & Conclusion

End-to-End Sequence Implemented in the Notebook

- 1

Analyze the hospital case study and identify its source systems
- 2

Create synthetic source-system datasets
- 3

Perform basic data-quality validation
- 4

Integrate all source systems using patient_id
- 5

Engineer operational and health-indicator features
- 6

Generate a demonstration risk score and risk level
- 7

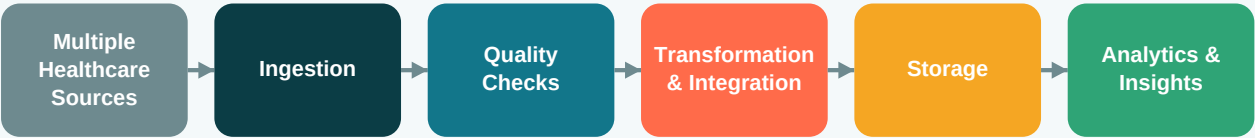
Store the processed data in SQLite, then read it back for analysis
- 8

Calculate waiting-time and patient-risk analytics
- 9

Generate charts and pipeline-monitoring metrics
- 10

Export the final analytics-ready dataset as CSV

Final Solution Architecture



This project demonstrates the core Data Engineering Fundamentals required by the case study. Multiple healthcare data sources are represented, validated and integrated into a unified dataset. The transformed data is stored in SQLite and used to produce waiting-time, risk and monitoring analytics. The workflow provides a clear foundation that could be extended with production ingestion tools, cloud storage, stream processing, live dashboards and clinically validated machine-learning models.

Student	Student ID
Chetan Kalagarla	2300033450

Reference note: the architecture diagrams are simplified, project-specific representations based on standard healthcare data-pipeline patterns, included to visually explain the structure of this implementation.